Prompt:
Write a recursive, object-oriented C++ program (use classes/objects, not free functions) that solves Sudoku puzzles with brute-force depth-first search and backtracking. Rules: a 9x9 grid made of nine 3x3 blocks. Every cell holds a digit 1-9, and each digit must appear exactly once in every row, column, and block. Algorithm: 1. Find an empty cell (k of 81 cells unfilled). 2. Compute its candidates: digits not already used in its row, column, or block. 3. If there are no candidates, return "no solution" (backtrack). 4. Otherwise, place each candidate in turn and recurse with k+1 cells filled. 5. If every candidate fails, backtrack. A solution is found when all 81 cells are filled. 6. Don't stop at the first solution: find and print ALL solutions, since a puzzle may have more than one. Input: five files, puzzle1.txt through puzzle5.txt. Each holds a 9x9 grid in which digits are given in cells and "_" is a blank. (solution1.txt is the known solution to puzzle1.txt, for debugging.) Output, for each puzzle file: - The file name (e.g., "puzzle1.txt") - The original puzzle, printed - Every solution, printed, or "No solution found" if none exists. Deliver: a well-structured design (e.g., a Puzzle/Grid class and a Solver class), complete compilable code, and brief build/run instructions.
2: AI analysis:

a) Claude by Anthropic and Gemini by Google.
b) Accessed via Google search
c) The prompt used is above.
d) Raw code for both is contained in .txt files on GitHub.
e) Maintainability: Gemini wins; Claude's output contains some utterly bizarre naming conventions; for example, `isConsistent() const { return consistent_; }`. Or contextless comments like `// keep it so the original`. Gemini, on the other hand, gives mostly useless generic comments but uses a FAR more readable naming convention. Correctness: Claude loses because it violates object-oriented principles; a large portion of the grid class code is hardcoded. Claude's output also isn't even close to running at all and crashes when attempting to compile by itself. For time complexity, both have a worst case of O(N^N^2) as a result of the massive number of calls to brute-force each square. The space complexity is n^2 since that is the maximum number of calls that will exist on the stack in either case.
f) I chose Gemini since the base code, while needing major improvements, is baseline functional and readable. This creates a far better starting point for improvement.
g) The first fix I made was changing how the output was printed to terminal to align with assignment standards, see solution1.txt. Next I rewrote half of the code to merge the 2 classes, the approach of having them separate only makes the code messier so I merged their functionality and made Solve a member operation of grid. Removed IsEmpty, getCell, setCell since their functionality was useless now. Rewrote the entire recursive solution and set based filtering mechanism to work as member functions of the Grid. Doing this cut down the solution by around half the lines of code. This makes code dramatically more readable. The syntax is far simpler since the number of calls made between class objects is reduced dramatically only seeing this when main makes calls. The code is now dramatically more maintainable.